

Refatorando o sistema de pedidos da loja acadêmica

Um relato da aplicação dos princípios de projeto

Pedro

Como cheguei nisso

Peguei o projeto `ProjetoPrincipiosDesignJava` e, como o enunciado avisava, ele funcionava direitinho – dava para cadastrar cliente, montar pedido, calcular desconto, pagar e ver o resumo. O problema não estava no “funciona ou não funciona”, mas em como o código estava organizado por baixo do capô: uma classe fazendo coisa demais, uma interface obrigando todo mundo a implementar método que não usa, uma herança que não fazia o menor sentido e por aí vai.

Segui o roteiro de sete etapas e, para cada uma, tentei primeiro entender por que aquele trecho incomodava antes de sair mexendo. Documento aqui o raciocínio de cada etapa: o que vi de errado, por que isso é um problema de verdade (e não só “feio”), o que eu fiz e como confirmei que nada quebrou. Em nenhum momento removi uma funcionalidade – o objetivo era deixar a estrutura mais sadia, mantendo o comportamento observável idêntico ao original.

1 Etapa 1 — Tirando peso do PedidoService (Responsabilidade Única)

O método `finalizarPedido` fazia praticamente tudo sozinho: calculava o total, escrevia num arquivo, montava um resumo e disparava o pagamento. Repara no trecho:

```
public void finalizarPedido(Pedido pedido, String formaPagamento) {  
    ...  
    Files.writeString(  
        Path.of("pedidos.txt"),  
        linha,  
        StandardOpenOption.CREATE,  
        StandardOpenOption.APPEND  
    );  
    ...  
}
```

O que me incomodou aqui não foi só a quantidade de linhas, mas o fato de que essa classe passou a ter dois motivos completamente diferentes para mudar: se eu decidir trocar o arquivo texto por um banco de dados, vou mexer na mesma classe que decide desconto e forma de pagamento. Isso é o cheiro clássico de responsabilidade única sendo violada.

A saída foi tirar essa parte e colocar numa classe própria, `PedidoRepository`, cuja única razão de existir é salvar o pedido. O `PedidoService` passou só a chamar essa classe, sem saber que por trás disso tem `Files` e `Path`:

```
public class PedidoRepository {  
    public void salvar(Pedido pedido, double total) {  
        String linha = pedido.getCliente().getNome() + ";" + total
```

```

        + System.lineSeparator();
        Files.writeString(ARQUIVO_PEDIDOS, linha,
            StandardOpenOption.CREATE, StandardOpenOption.APPEND);
    }
}

// dentro de PedidoService:
pedidoRepository.salvar(pedido, total);

```

Rodei de novo depois disso e o arquivo `pedidos.txt` continuou sendo gerado normalmente, com a mesma linha de antes.

2 Etapa 2 — Separando a interface de pagamento (Segregação de Interfaces)

Aqui o problema pulou aos olhos assim que abri as três classes de pagamento lado a lado. A interface `IPagamento` exigia `pagar`, `parcelar` e `gerarBoleto` de todo mundo, mas o PIX, por exemplo, não pode ser parcelado – e o jeito que isso foi resolvido foi lançando uma exceção:

```

public interface IPagamento {
    void pagar(double valor);
    void parcelar(double valor, int parcelas);
    void gerarBoleto(double valor);
}

// em PagamentoPix:
public void parcelar(double valor, int parcelas) {
    throw new UnsupportedOperationException("PIX nao pode ser parcelado
        .");
}

```

Isso é um sinal de interface grande demais: a classe é obrigada a "assinar" um contrato que não consegue cumprir de verdade. Na prática, isso é uma bomba-relógio – um dia alguém vai chamar `parcelar` num PIX achando que é seguro, porque o compilador deixa, e a aplicação vai quebrar em produção.

Quebrei a interface em três menores, cada uma com uma única operação, e deixei cada classe implementar só o que ela realmente faz:

```

public interface IPagavel { void pagar(double valor); }
public interface IParcelavel { void parcelar(double valor, int parcelas
    ); }
public interface IGeradorBoleto { void gerarBoleto(double valor); }

public class PagamentoPix implements IPagavel { ... }
public class PagamentoCartao implements IPagavel, IParcelavel { ... }
public class PagamentoBoleto implements IPagavel, IGeradorBoleto { ...
    }

```

Agora nem compila se alguém tentar chamar `parcelar` num objeto do tipo `PagamentoPix` – o erro que antes só aparecia em tempo de execução vira um erro de compilação, que é bem mais barato de corrigir.

3 Etapa 3 — Tirando uma herança que não fazia sentido (Composição em vez de herança)

Essa foi rápida de identificar, mas curiosa de entender o motivo de existir: `PedidoService` `extends` `PagamentoCartao`. Um serviço de pedidos sendo, ao mesmo tempo, uma forma de pagamento por cartão? Não faz sentido nenhum, e olhando o código percebi que nenhum método herdado de `PagamentoCartao` era realmente usado por `PedidoService` – a herança estava lá sem propósito.

```
public class PedidoService extends PagamentoCartao {
    ...
}
```

Herança deveria representar uma relação de "é um". `PedidoService` não é um `PagamentoCartao`; ele apenas *usa* uma forma de pagamento para concluir o pedido. Fiz a troca para composição: `PedidoService` passou a receber a forma de pagamento como um objeto colaborador, e não mais como uma superclasse.

```
public class PedidoService {
    // sem "extends PagamentoCartao"
    public void finalizarPedido(Pedido pedido, CalculadoraDesconto
                               desconto,
                               IPagavel formaPagamento) {
        ...
        formaPagamento.pagar(total);
    }
}
```

Isso também deixou o caminho livre para resolver a etapa 6, já que agora dá para passar qualquer forma de pagamento sem depender de uma classe específica.

4 Etapa 4 — Encurtando o caminho até a cidade (Lei de Demeter)

Para descobrir a cidade de entrega, o `PedidoService` precisava atravessar uma cadeia inteira de objetos:

```
public String obterCidadeEntrega(Pedido pedido) {
    return pedido.getCliente().getEndereco().getCidade().getNome();
}
```

O nome bonito para isso é "violação da Lei de Demeter", mas na prática o problema é bem concreto: o `PedidoService` precisa saber que um `Cliente` tem `Endereco`, que um `Endereco` tem `Cidade`, e que `Cidade` tem um nome. Se um dia `Endereco` ganhar, por exemplo, um objeto `Pais` no meio do caminho, essa linha quebra – e ela está numa classe que não tem nada a ver com endereçamento.

A ideia da correção é simples: cada classe esconde sua própria estrutura interna e só expõe o que interessa para quem está de fora. `Endereco` passou a saber devolver diretamente o nome da cidade, e `Cliente` passou a saber devolver diretamente sua cidade de entrega:

```
// Endereco
public String getNomeCidade() { return cidade.getNome(); }

// Cliente
```

```

public String getCidadeEntrega() { return endereco.getNomeCidade(); }

// PedidoService
public String obterCidadeEntrega(Pedido pedido) {
    return pedido.getCliente().getCidadeEntrega();
}

```

O `PedidoService` conversa só com o seu "vizinho direto" (`Cliente`) e não precisa mais saber o que existe dentro dele.

5 Etapa 5 — Tirando o desconto do meio do `PedidoService` (Aberto/Fechado e Inversão de Dependência)

O cálculo de desconto era uma sequência de `if/else` comparando uma `String` com o tipo do cliente:

```

if (tipoCliente.equals("ALUNO")) {
    total *= 0.90;
} else if (tipoCliente.equals("PROFESSOR")) {
    total *= 0.85;
} else if (tipoCliente.equals("FUNCIONARIO")) {
    total *= 0.80;
}

```

Pensando no enunciado ("e se aparecer um novo tipo de cliente amanhã?"), ficou claro o problema: cada tipo novo de cliente ia exigir voltar nessa mesma classe e adicionar mais um `else if`. Isso é o oposto do princípio Aberto/Fechado, que diz que eu deveria conseguir estender o comportamento sem precisar reabrir uma classe que já está pronta e testada.

Criei uma abstração para o cálculo do desconto e uma implementação para cada tipo de cliente. O `PedidoService` não sabe mais que existe um "ALUNO" ou um "PROFESSOR" – ele só sabe que recebe algo capaz de aplicar um desconto:

```

public interface CalculadoraDesconto { double aplicar(double total); }

public class DescontoAluno implements CalculadoraDesconto {
    public double aplicar(double total) { return total * 0.90; }
}

// PedidoService
public double calcularTotal(Pedido pedido, CalculadoraDesconto desconto) {
    double total = ... soma dos itens ...;
    return desconto.aplicar(total);
}

```

Se amanhã precisar de um desconto para "EGRESSO", crio uma classe nova que implementa `CalculadoraDesconto` e pronto – o `PedidoService` nem precisa ser recompilado.

6 Etapa 6 — Tirando o if/else de pagamento (Aberto/Fechado e Inversão de Dependência)

O mesmo padrão de problema da etapa anterior aparecia de novo em `finalizarPedido`, só que para escolher a forma de pagamento:

```
if (formaPagamento.equals("CARTAO")) {
    PagamentoCartao pagamento = new PagamentoCartao();
    pagamento.pagar(total);
} else if (formaPagamento.equals("PIX")) {
    ...
} else if (formaPagamento.equals("BOLETO")) {
    ...
}
```

Como já tinha resolvido a etapa 3 usando composição, a solução aqui ficou natural: em vez de `PedidoService` receber uma `String` e decidir sozinho qual classe instanciar, ele passou a receber a forma de pagamento já pronta, do tipo `IPagavel` (a interface criada lá na etapa 2). Quem decide qual implementação usar é quem chama o serviço, não o serviço em si:

```
public void finalizarPedido(Pedido pedido, CalculadoraDesconto desconto
,
                           IPagavel formaPagamento) {
    double total = calcularTotal(pedido, desconto);
    ...
    formaPagamento.pagar(total);
}

// em Main.java:
IPagavel formaPagamento = new PagamentoCartao();
servico.finalizarPedido(pedido, desconto, formaPagamento);
```

Os `if/else` sumiram de vez. Uma forma de pagamento nova (um "carteira digital", por exemplo) vira só mais uma classe implementando `IPagavel`.

7 Etapa 7 — O problema escondido em `EntregaRetiradaLoja` (Substituição de Liskov)

Essa foi a etapa que mais me fez parar para pensar. `EntregaRetiradaLoja` herdava de `Entrega`, mas fazia uma coisa que a classe-mãe nunca fazia: lançar uma exceção.

```
public class EntregaRetiradaLoja extends Entrega {
    @Override
    public double calcularFrete(double total) {
        if (total < 50.0) {
            throw new IllegalStateException(
                "Retirada na loja disponivel apenas para pedidos a
                partir de R$ 50,00.");
        }
        return 0.0;
    }
}
```

O princípio da substituição de Liskov diz que eu deveria poder trocar um `Entrega` por um

EntregaRetiradaLoja em qualquer lugar do código sem que nada de estranho aconteça. Só que aqui não é bem assim: qualquer código que recebesse uma **Entrega** e chamasse **calcularFrete** esperando um número de volta corria o risco de tomar uma exceção que a classe base jamais lançaria. É exatamente o tipo de surpresa que esse princípio existe para evitar.

Resolvi de duas formas ao mesmo tempo: tirei a herança (voltando ao tema da etapa 3, composição em vez de herança) e mudei o contrato para deixar claro, de antemão, quando uma forma de entrega está ou não disponível – sem depender de exceção para isso. Quando a retirada gratuita não é possível, a classe usa uma **Entrega** comum por dentro, como alternativa:

```
public interface IEntrega {
    boolean estaDisponivel(double total);
    double calcularFrete(double total);
}

public class EntregaRetiradaLoja implements IEntrega {
    private final IEntrega entregaAlternativa;
    public EntregaRetiradaLoja() { this(new Entrega()); }
    public EntregaRetiradaLoja(IEntrega entregaAlternativa) {
        this.entregaAlternativa = entregaAlternativa;
    }
    public boolean estaDisponivel(double total) { return total >= 50.0; }
    public double calcularFrete(double total) {
        if (estaDisponivel(total)) return 0.0;
        return entregaAlternativa.calcularFrete(total);
    }
}
```

Agora **calcularFrete** nunca lança exceção, para nenhuma das duas implementações – a substituição deixou de ser um risco. Quem quiser saber com antecedência se a retirada grátis vale para aquele pedido pode simplesmente perguntar com **estaDisponivel(total)**.

8 Conferindo que nada quebrou

Depois de cada etapa, recompilei e rodei o projeto de novo (**javac + java**) só para ter certeza de que não tinha introduzido nenhum efeito colateral. No fim, testei especificamente:

- o cálculo do pedido de exemplo, que continua dando R\$ 144,00 com desconto de aluno;
- o pagamento nos três meios (cartão, PIX e boleto), cada um imprimindo a mensagem esperada;
- a apresentação da cidade de entrega, do cliente e do total, tudo igual ao original;
- o caso de borda de **EntregaRetiradaLoja** com um pedido abaixo de R\$ 50 – antes derrubava a aplicação com uma exceção, agora devolve um frete alternativo válido.

A saída do **Main.java** ficou idêntica à versão original em tudo que é observável – o que muda é só a estrutura por trás.

9 O que eu levo dessa refatoração

Quais classes ficaram com responsabilidades mais claras? O **PedidoService** foi quem mais mudou de perfil: antes sabia demais sobre como salvar em arquivo, qual desconto dar para cada tipo de cliente e qual classe de pagamento instanciar. Agora ele só orquestra o fluxo, apoiado

em abstrações (`PedidoRepository`, `CalculadoraDesconto`, `IPagavel`). `Cliente` e `Endereco` também ficaram mais "donos" da própria informação, em vez de deixar tudo exposto para quem estiver de fora mexer.

Quantas classes precisariam mudar para um novo tipo de pagamento ou desconto?

Antes da refatoração, qualquer novidade desse tipo ia bater direto no `PedidoService`. Depois, basta criar uma classe nova implementando `CalculadoraDesconto` ou `IPagavel` – nenhuma linha do `PedidoService` precisa ser tocada. É exatamente o resultado que o princípio Aberto/Fechado promete.